

CSCI E-106: Assignment 4

Due Date: February 27, 2026 at 11:59 pm EST

Problem 1

Refer to Sales growth Data.

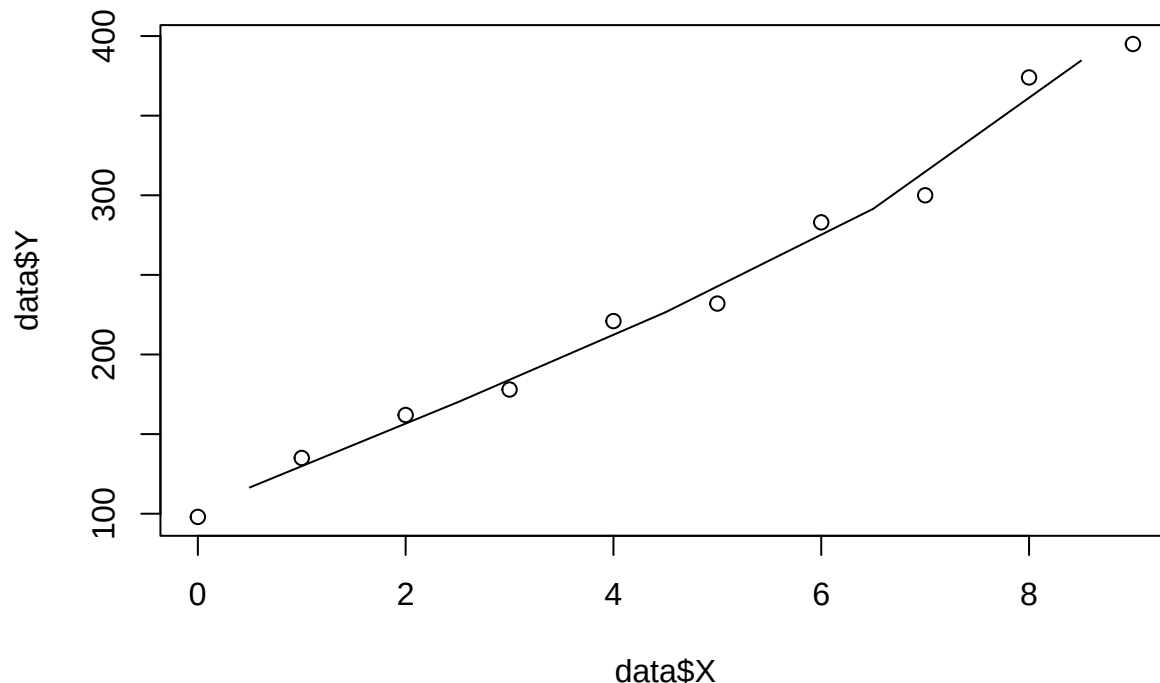
```
set.seed(1023)
sales_growth_data <- read.csv("Sales Growth Data.csv")
data = sales_growth_data
```

a-) Divide the range of the predictor variable (coded years) into five bands of width 2.0, as follows: Band 1 ranges from $X = -.5$ to $X = 1.5$; band 2 ranges from $X = 1.5$ to $X = 3.5$; and so on. Determine the median value of X and the median value of Y in each band and develop the band smooth by connecting the five pairs of medians by straight lines on a scatter plot of the data.

```
getMedians <- function(data, width, startX) {
  endX = startX + width
  selection = data[data$X > startX & data$X < endX,]
  return(c(X=median(selection$X), Y=median(selection$Y)))
}

par(mfrow=c(1,1))
plot(data$X, data$Y)

medians = getMedians(data,2,-0.5)
for(i in 1:4) {
  nextMedians = getMedians(data,2,-0.5+2*i)
  segments(medians[["X"]], medians[["Y"]], nextMedians[["X"]], nextMedians[["Y"]])
  medians = nextMedians
}
```



Does the band smooth suggest that the regression relation is linear? Discuss.

The regression relation for this data is clearly linear. Generally, there are examples of non-linear data that would have band smooth. For instance, high frequency changes in Y will be smooth for some band widths and jagged for others. Data with concentrated values can appear to have medians higher or lower than the means, which can smooth or unsmooth the connections.

Data that is linear may tend in general to have band smooth.

b-) Create a series of seven overlapping neighborhoods of width 3.0 beginning at $X = -.5$. The first neighborhood will range from $X = -.5$ to $X = 2.5$; the second neighborhood will range from $X = .5$ to $X = 3.5$; and so on. For each of the seven overlapping neighborhoods, fit a linear regression function and obtain the fitted value $\hat{Y}_c$ at the center X_c of the neighborhood. Develop a simplified version of the lowess smooth by connecting the seven $(X_c, \hat{Y}_c)$ pairs by straight lines on a scatter plot of the data.

```
plotB = function(data) {
  getCenter <- function(data, width, startX) {
    midX = startX + (width/2)
    endX = startX + width
    selection = data[data$X > startX & data$X < endX,]

    selectionFit = lm(Y~X, selection)
    res = data.frame(X=c(midX))
    res$Y = predict(selectionFit, res)
    return(res)
  }

  plot(data$X, data$Y)

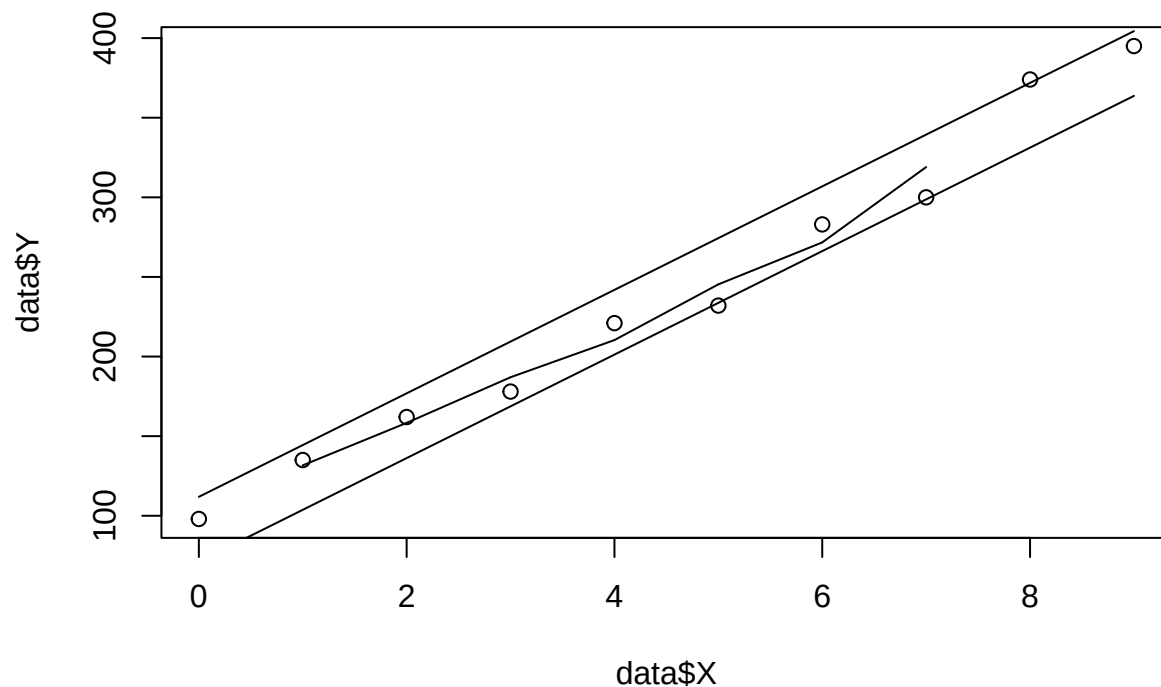
  center = getCenter(data, 3, -0.5)
  for(i in 1:6) {
    nextCenter = getCenter(data, 3, -0.5 + i)
    segments(center[["X"]], center[["Y"]], nextCenter[["X"]], nextCenter[["Y"]])
    center = nextCenter
  }
}
```

```
}
}
```

c-) Obtain the 95 percent confidence band for the true regression line and plot it on the plot prepared in part

```
lm_ = lm(Y~X, data)
ci = data.frame(X=c(min(data$X), max(data$X)))
ci = cbind(ci, predict(lm_, ci, interval="confidence"))

plotB(data)
segments(ci[1,"X"], ci[1,"lwr"], ci[2,"X"], ci[2,"lwr"])
segments(ci[1,"X"], ci[1,"upr"], ci[2,"X"], ci[2,"upr"])
```



(b). Does the simplified loess smooth fall entirely within the confidence band for the regression line?

Yes.

What does this tell you about the appropriateness of the linear regression function?

Because the simplified loess smooth is a closer fit to the data, it would tend to exceed the confidence band for data that doesn't follow a linear regression. That the simplified loess smooth falls inside of the confidence band of the linear regression indicates that the linear regression matches the closer fit, that the closer fit is close to linear, and therefore the linear regression is a good fit for the data.

Problem 2

Using the ozone data under faraway library, if you are using the R Studio, please use the command below to download the data:

```
library(faraway)
data(ozone, package="faraway")
data = ozone
```

Use O3 as the response and temp, humidity and ibh as predictors to answer the questions below.

```
data = data[, c("O3", "temp", "humidity", "ibh")]
k = 3
```

a-) Create train and test data sets. Use 70set and use remaining data (30set.seed(1023)).

```
popN = nrow(data)
alphaN = 7/10*popN
idx_train = sample(popN, size = alphaN, replace=FALSE)
train = data[idx_train,]
trainN = nrow(train)
test = data[-idx_train,]
testN = nrow(test)
```

b-) Build a regression model to predict O3 on the train data set. Write down the regression model and comment on the model performance.

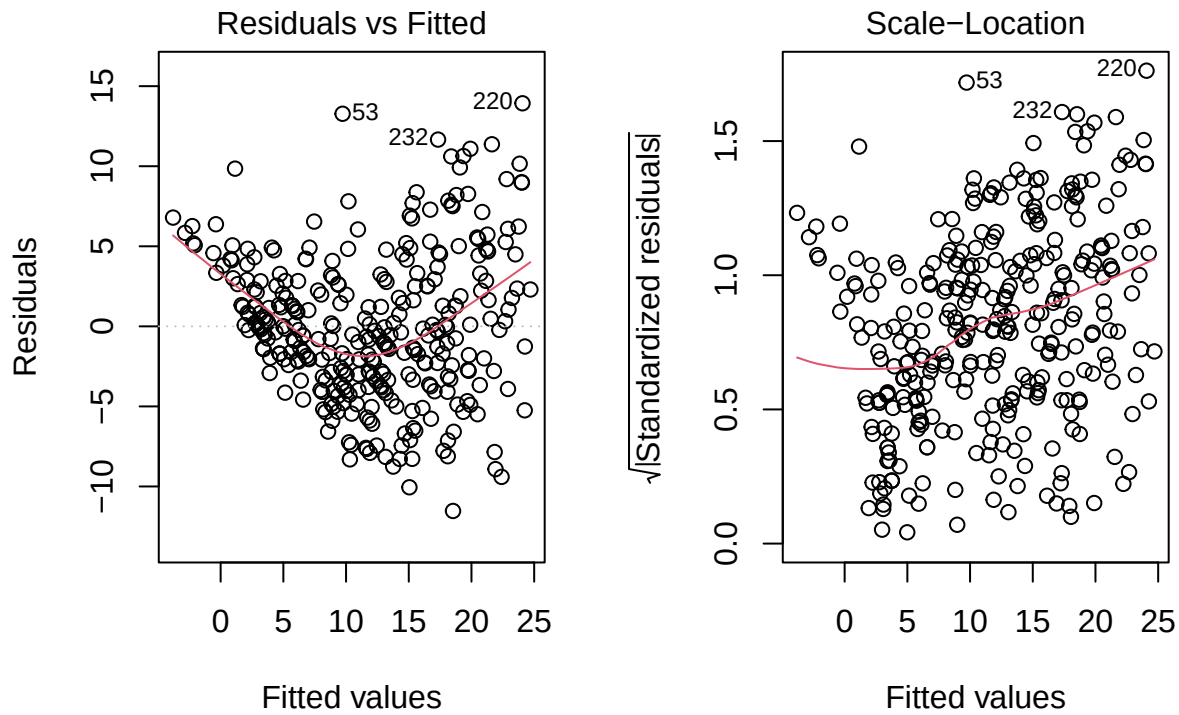
```
lm_ = lm(O3 ~ ., data)
lm_summary = summary(lm_)
print(lm_summary)
```

```
##
## Call:
## lm(formula = O3 ~ ., data = data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -11.5291  -3.0137  -0.2249   2.8239  13.9303
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.049e+01  1.616e+00  -6.492 3.16e-10 ***
## temp         3.296e-01  2.109e-02  15.626 < 2e-16 ***
## humidity     7.738e-02  1.339e-02   5.777 1.77e-08 ***
## ibh          -1.004e-03  1.639e-04  -6.130 2.54e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.524 on 326 degrees of freedom
## Multiple R-squared:  0.684, Adjusted R-squared:  0.6811
## F-statistic: 235.2 on 3 and 326 DF, p-value: < 2.2e-16
```

The P-values are all significantly less than 0.05.

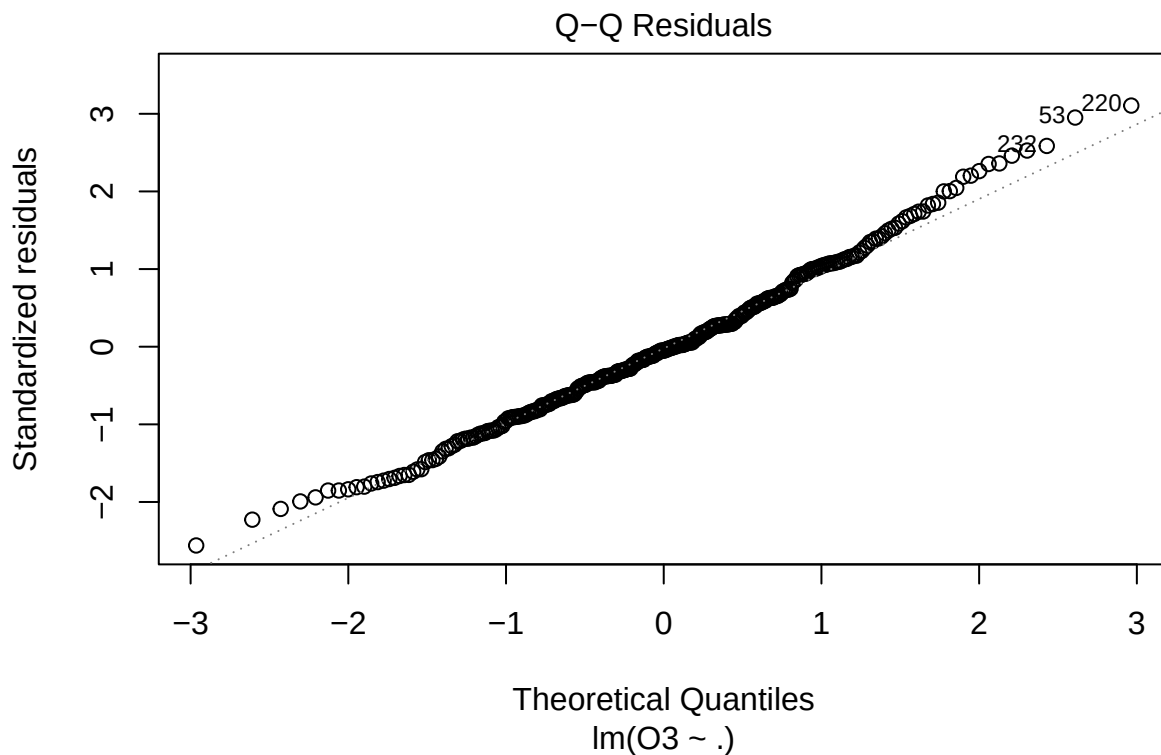
c-) Visually by using the graphs, comments on regression model assumptions.

```
par(mfrow=c(1,2))
plot(lm_, which=c(1,3))
```



The residuals increase toward the higher extrema. The residuals are unevenly distributed at the lower extreme.

```
plot(lm_, which=2)
```



The Q-plot has deviation at the ends.

d-) Test the model performance on the test data and comment on model stability and performance.

```
predictTest <- predict(lm_, test)
modelTest <- cbind(predictTest, test)
```

```

testResiduals = modelTest$O3 - modelTest$predictTest
dimE = testN - k - 1

sst = sum((modelTest$O3 - mean(modelTest$O3))^2)
sse = sum(testResiduals^2)
ssr = sum((modelTest$predictTest - mean(modelTest$O3))^2)

mse = sse/dimE
rse = sqrt(mse)
rmse = sqrt(sse/length(testResiduals))
rSquared = 1.0 - (sse/sst)
mae = sum(testResiduals)/length(testResiduals)

model_summary = data.frame(MSE=c(mse), RSE=rse, RMSE=c(rmse),
                           Rsquared=c(rSquared), MAE=c(mae))
print(model_summary)

```

```

##          MSE          RSE          RMSE Rsquared          MAE
## 1 24.43049 4.942721 4.842858 0.633201 0.03392619

```

The R^2 in the training set was 0.684. In the test it is 0.633.

e-) *Perform Breusch-Pagan Test, write down null and alternative hypotheses and your conclusion.*

H_0 : The error variances in the regression residuals are homoscedastic.

H_a : The error variances in the regression residuals are heteroscedastic.

```
bptest(lm_)
```

```

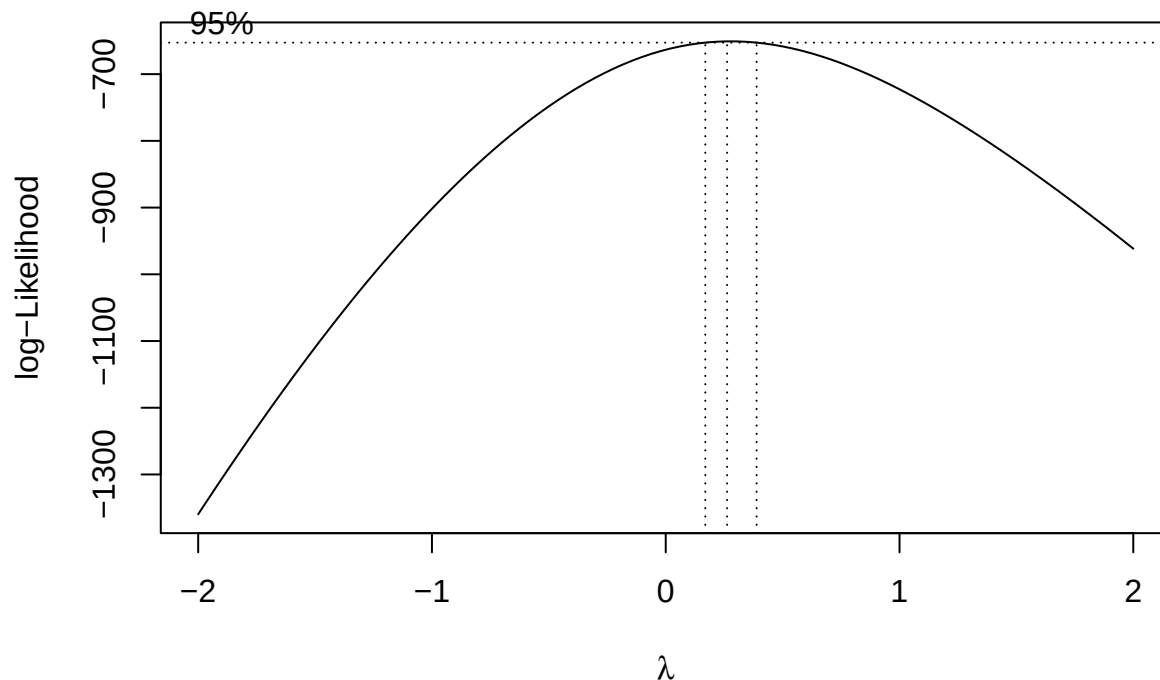
##
## studentized Breusch-Pagan test
##
## data:  lm_
## BP = 32.818, df = 3, p-value = 3.519e-07

```

In the linear model, the p-value is small, so we accept the alternative hypothesis. The residual variances are heteroscedastic.

f-) *Use the Box-Cox method to determine the best transformation on the response.*

```
bc_model = MASS::boxcox(lm_, plotit=TRUE)
```



```
lambda_bc = bc_model$x[which.max(bc_model$y)]
```

The critical lambda value is at 0.263.

```
train$O3_bc = (train$O3^lambda_bc-1)/lambda_bc
lm_bc_ = lm(O3_bc ~ . - O3, train)
lm_bc_summary = summary(lm_bc_)
print(lm_bc_summary)
```

```
##
## Call:
## lm(formula = O3_bc ~ . - O3, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.13306 -0.39203  0.05344  0.49347  1.29835
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -7.781e-01  2.970e-01  -2.619  0.00941 **
## temp         5.823e-02  3.898e-03  14.938 < 2e-16 ***
## humidity     1.363e-02  2.327e-03   5.857 1.65e-08 ***
## ibh          -1.892e-04  2.938e-05  -6.437 7.20e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6853 on 226 degrees of freedom
## Multiple R-squared:  0.7417, Adjusted R-squared:  0.7382
## F-statistic: 216.3 on 3 and 226 DF,  p-value: < 2.2e-16
```

The R^2 in the original training set was 0.684. In the transformed data it is 0.742.

If transformation is needed, perform the transformation and rebuild the model on the train data and check the model stability on the test data set.

```

predictTest <- (lambda_bc * predict(lm_bc_, test) + 1)^(1/lambda_bc)
modelTest <- cbind(predictTest, test)
testResiduals = modelTest$O3 - modelTest$predictTest
dimE = testN - k - 1

sst = sum((modelTest$O3 - mean(modelTest$O3))^2)
sse = sum(testResiduals^2)
ssr = sum((modelTest$predictTest - mean(modelTest$O3))^2)

mse = sse/dimE
rse = sqrt(mse)
rmse = sqrt(sse/length(testResiduals))
rSquared = 1.0 - (sse/sst)
mae = sum(testResiduals)/length(testResiduals)

model_summary = data.frame(MSE=c(mse), RSE=rse, RMSE=c(rmse),
                           Rsquared=c(rSquared), MAE=c(mae))
print(model_summary)

```

```

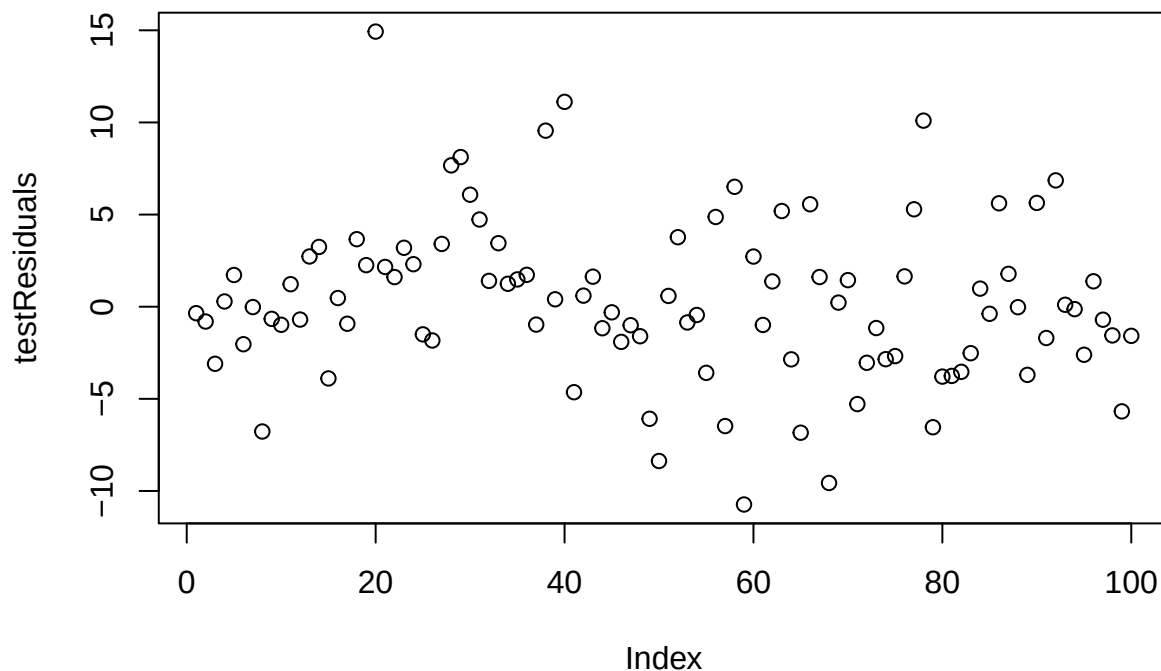
##      MSE      RSE      RMSE Rsquared      MAE
## 1 19.4613 4.411496 4.322366 0.7078084 0.3041547

```

In the transformed training data, R^2 is 0.742.

In the transformed test data, R^2 is 0.708.

```
plot(testResiduals)
```



Problem 3

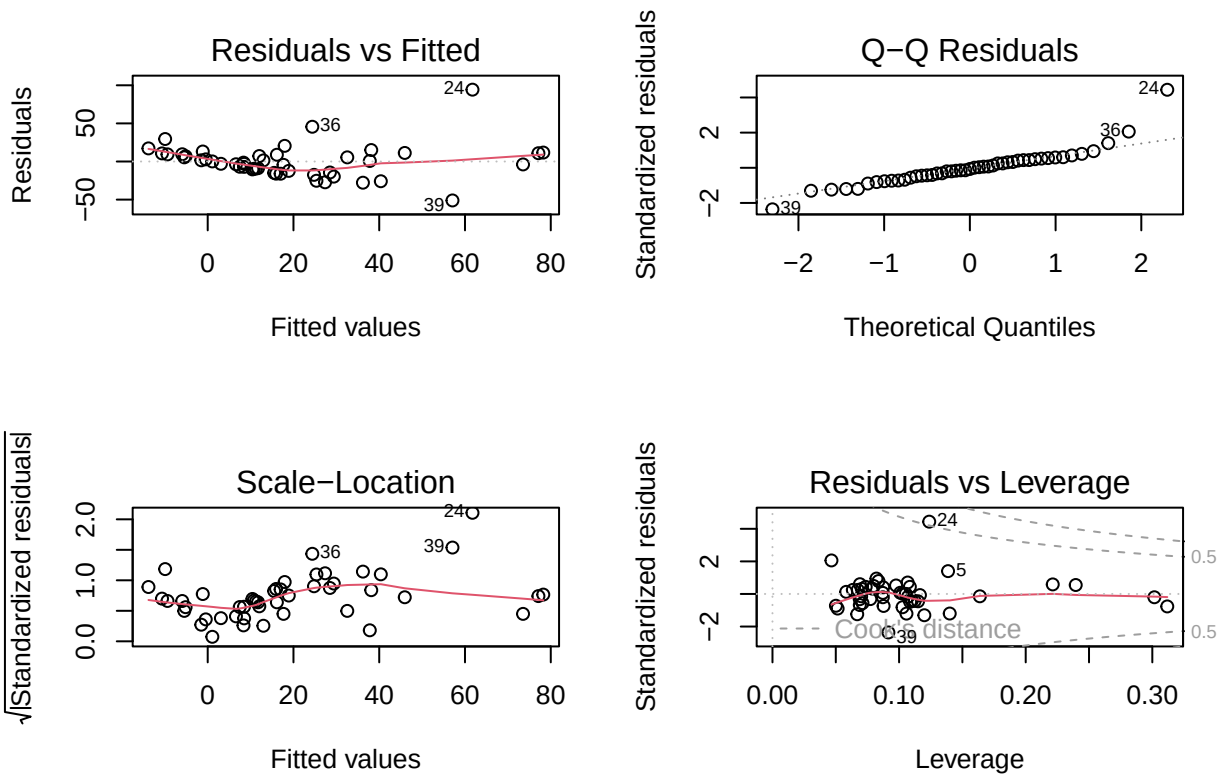
Using the *teengamb* data under *faraway* library, if you are using the R Studio, please use the command below to download the data:


```
library(faraway)
data(teengamb, package="faraway")
```

a-) *Fit a model with gamble as the response and the other variables as predictors.*

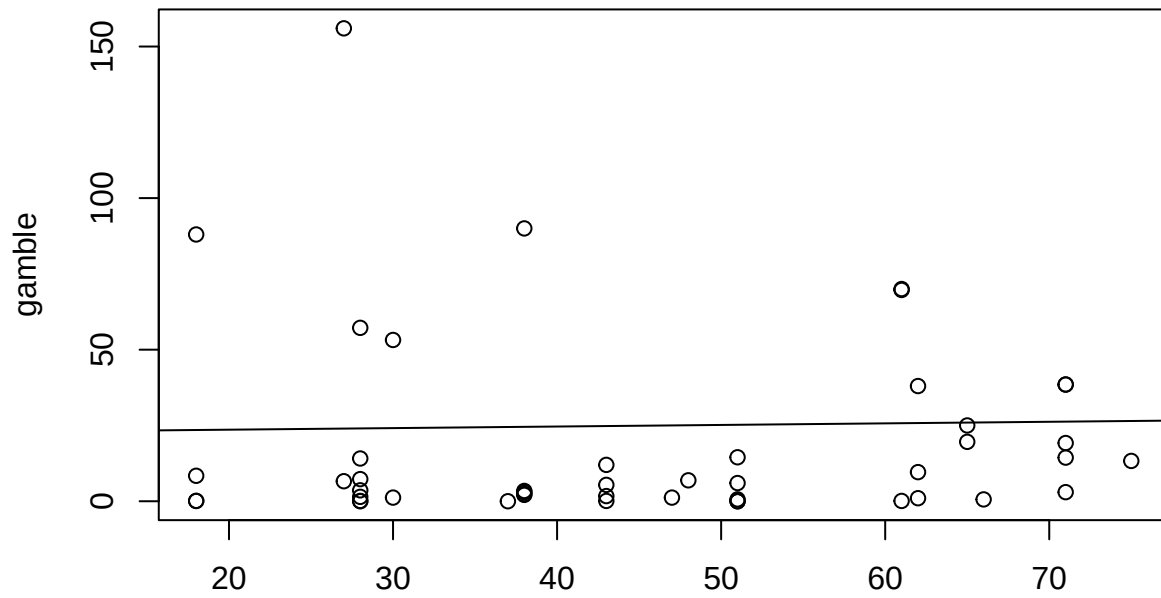
```
lm_ = lm(gamble ~ ., teengamb)
lm_summary = summary(lm_)
print(lm_summary)
```

```
##
## Call:
## lm(formula = gamble ~ ., data = teengamb)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -51.082 -11.320  -1.451   9.452  94.252
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  22.55565   17.19680   1.312   0.1968
## sex          -22.11833    8.21111  -2.694   0.0101 *
## status         0.05223    0.28111   0.186   0.8535
## income         4.96198    1.02539   4.839 1.79e-05 ***
## verbal        -2.95949    2.17215  -1.362   0.1803
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 22.69 on 42 degrees of freedom
## Multiple R-squared:  0.5267, Adjusted R-squared:  0.4816
## F-statistic: 11.69 on 4 and 42 DF,  p-value: 1.815e-06
par(mfrow=c(2,2))
plot(lm_)
```

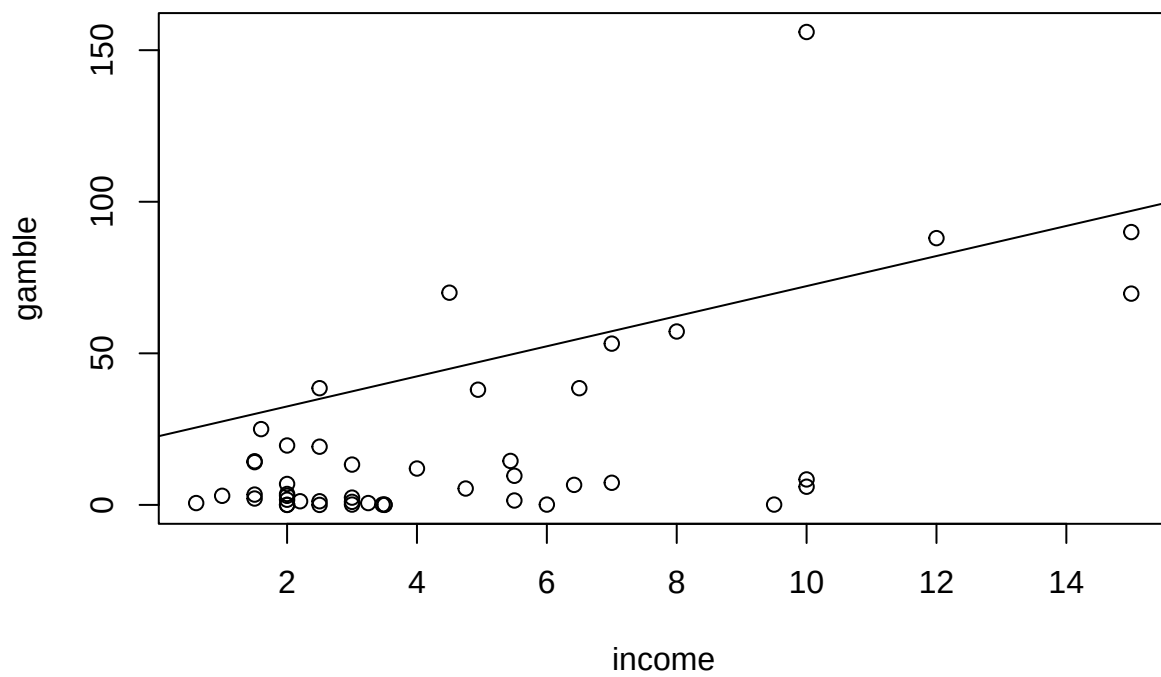


```
par(mfrow=c(1,1))
predictors = c("status", "income", "verbal")
for (predictor in predictors) {
  plot(teengamb[,predictor], teengamb[, "gamble"], main=paste(predictor, " vs gamble"), xlab=predictor, ylab="gamble")
  abline(lm_1$coefficients[["(Intercept)"]], lm_1$coefficients[[predictor]])
}
```

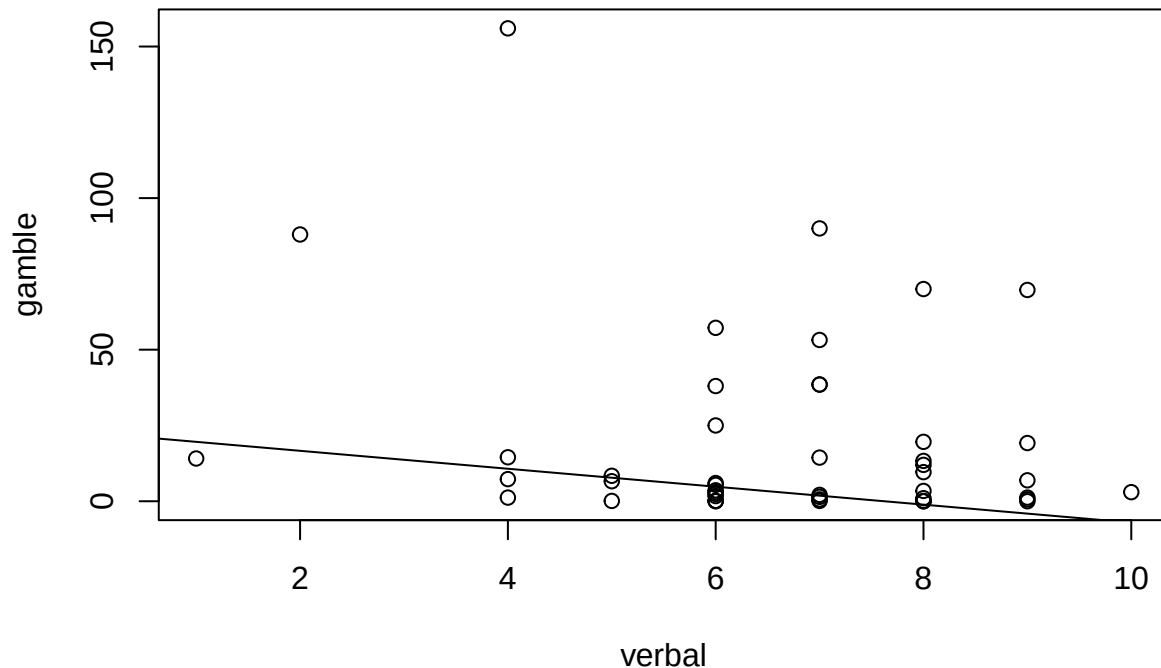
status vs gamble



income vs gamble



verbal vs gamble



Using the graphs, comment on the regression model and assumptions. From the residual and sqrt residual plots, you can see there is a difference in variability from the low extreme, increasing at higher observed values. There is a higher concentration of data at the low extreme.

The Q-Q Residuals plot has deviation at the ends. Points 24, 36, and 39, seem like outliers.

From the plot of leverage, you can see that some data points have high leverage.

The model seems to be heteroscedastic.

b-) Predict the data(teengamb) amount that a male with average (given these data) status, income and verbal score would gamble along with an appropriate 95% CI.

```
mean_male_data = data.frame(
  sex=c(0),
  status=mean(teengamb$status),
  income=mean(teengamb$income),
  verbal=mean(teengamb$verbal))
mean_predictions = predict(lm_, mean_male_data, interval="confidence")
print(mean_predictions)
```

```
##          fit      lwr      upr
## 1 28.24252 18.78277 37.70227
```

c-) Repeat the prediction for a male with maximal values (for this data) of status, income and verbal score.

```
max_male_data = data.frame(
  sex=c(0),
  status=max(teengamb$status),
  income=max(teengamb$income),
  verbal=max(teengamb$verbal))
max_predictions = predict(lm_, max_male_data, interval="confidence")
print(max_predictions)
```

```
##          fit      lwr      upr
## 1 71.30794 42.23237 100.3835
```

Which CI is wider and why is this result expected?

The confidence interval for maximal values is wider. This would be expected, because a) the variance of the residuals increases at the upper extrema b) variability of predictions tends to increase as you move away from the center of the data.

d-) Fit a model with $\sqrt{\text{gamble}}$ as the response but with the same predictors.

```
teengamb_sqrt = cbind(
  gamble_sqrt=sqrt(teengamb$gamble),
  teengamb[,names(teengamb) != "gamble"]
)

lm_sqrt_ = lm(gamble_sqrt ~ ., teengamb_sqrt)
summary(lm_sqrt_)

##
## Call:
## lm(formula = gamble_sqrt ~ ., data = teengamb_sqrt)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.6606 -1.0961 -0.2564  0.9786  5.4178
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.97707     1.57947   1.885  0.06638 .
## sex          -2.04450     0.75416  -2.711  0.00968 **
## status         0.03688     0.02582   1.428  0.16057
## income         0.47938     0.09418   5.090 7.94e-06 ***
## verbal        -0.42360     0.19950  -2.123  0.03967 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.084 on 42 degrees of freedom
## Multiple R-squared:  0.5646, Adjusted R-squared:  0.5231
## F-statistic: 13.61 on 4 and 42 DF,  p-value: 3.362e-07
```

Now predict the response and give a 95% prediction interval for the individual in (a). Take care to give your answer in the original units of the response.

```
mean_sqrt_predictions = predict(lm_sqrt_, mean_male_data, interval="confidence")
print(mean_sqrt_predictions^2)

##          fit      lwr      upr
## 1 16.39864 10.1167 24.19037

max_sqrt_predictions = predict(lm_sqrt_, max_male_data, interval="confidence")
print(max_sqrt_predictions^2)

##          fit      lwr      upr
## 1 75.65038 36.32746 129.2364
```

e-) Repeat the prediction for the model in (d) for a female with status=20, income=1, verbal = 10.

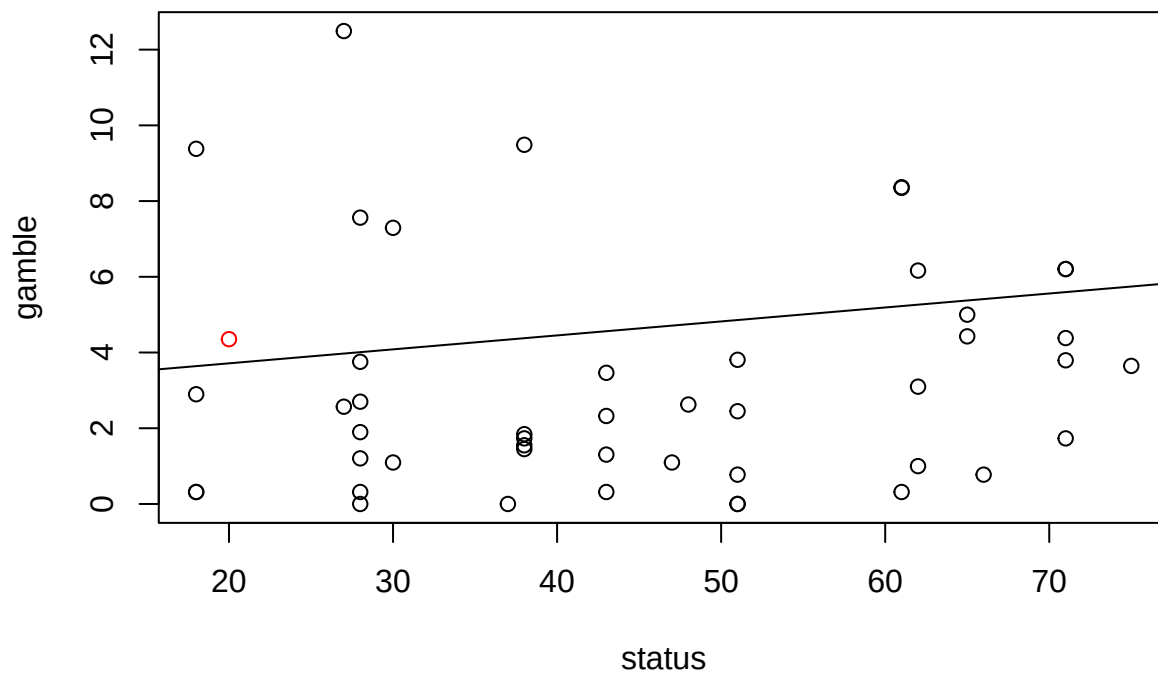
```
specific_female_data = data.frame(
  sex=1,
  status=20,
  income=1,
  verbal = 10
)
specific_sqrt_predictions = predict(lm_sqrt_, specific_female_data, interval="confidence")
print(specific_sqrt_predictions^2)
```

```
##          fit      lwr      upr
## 1 4.353398 19.76636 0.07451699
```

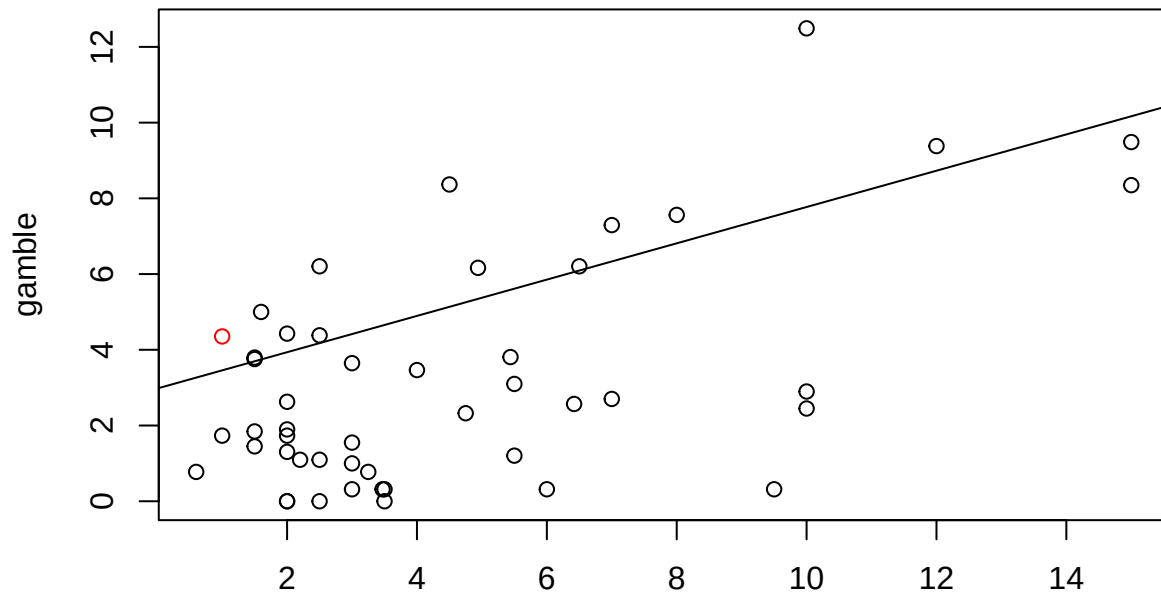
```
parallel = function(d, beta_0, beta_1) {
  beta0 + d*sqrt(1 + beta_1^2)
}
```

```
par(mfrow=c(1,1))
predictors = c("status", "income", "verbal")
for (predictor in predictors) {
  plot(teengamb_sqrt[,predictor], teengamb_sqrt[, "gamble_sqrt"], main=paste(predictor, " vs gamble"), xlab=predictor, ylab="gamble_sqrt")
  abline(lm_sqrt_$coefficients[["(Intercept)"]], lm_sqrt_$coefficients[[predictor]])
  points(specific_female_data[,predictor], specific_sqrt_predictions[, "fit"]^2, col="red")
}
```

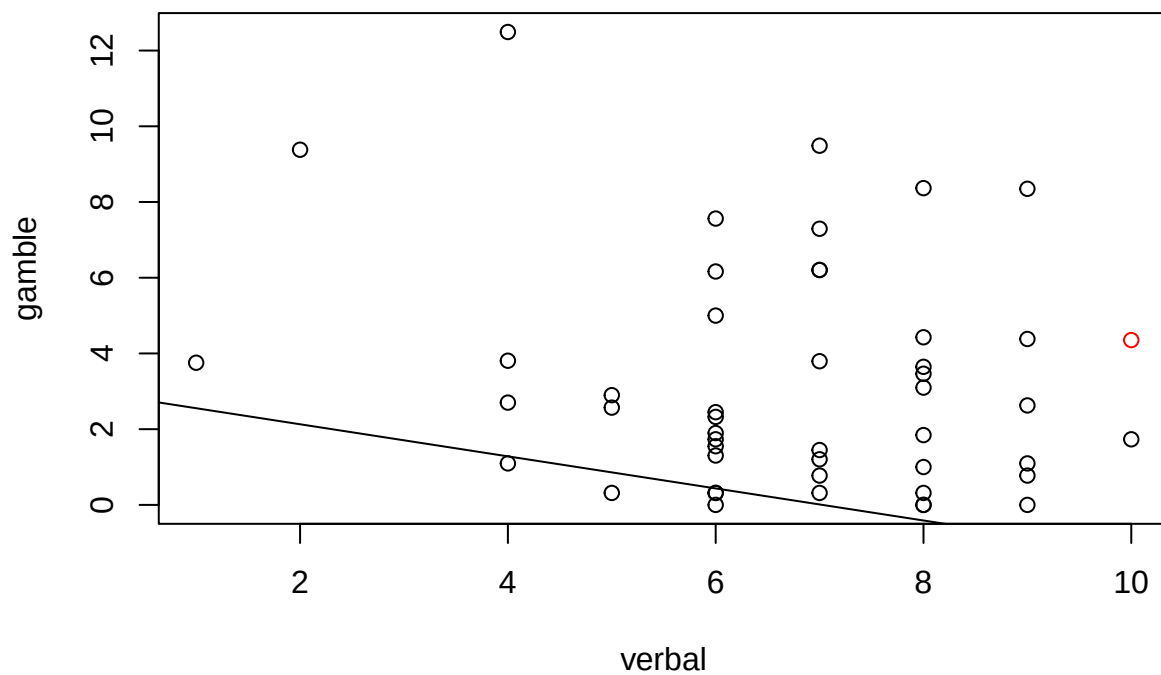
status vs gamble



income vs gamble



**income
verbal vs gamble**



Comment on the credibility of the result.

We see from Question 1 that looking at bands can

The predicted point seems to fall in the confidence interval of each of the plots. The most important predictor is income, and the prediction is taking place near the center of the income data.